

(Task 1 of 7) In this tutorial, we will learn even more about **mutable values**, illustrated with **arrays**.

0

(Task 2 of 7) Which choice best describes the heap at the end of the following program?

Python | 

(**Note:** we use @ddd (e.g., @123, @200, and @100) to represent heap addresses. Heap addresses are *random*. The numbers don't mean anything.)

Python [Run 

Pseudo

```
x = [ 2 ]    let x = vec[2]
x[0] = 33    x[0] = 33
print(x)     print(x)
```

2

- A. @100 = #(2); @200 = #(33)
- B. @200 = #(2)
- C. @200 = #(33)
- D. @200 = 33

@200 = #(33)

3

You gave the correct answer 🎉🎉🎉

4

Exactly one array was created. So, there must be at most one array on the heap. This rules out **A**.

For now, the heap maps addresses only to arrays. This rules out **D**.

The heap looks like **B** after evaluating `[ 2 ]`. However, the subsequent mutation changes the array. So, the correct answer is **C**.

(Task 3 of 7) Which choice best describes the heap at the end of the following program?

Python | 

Python [Run 

Pseudo

```
m = [ 1, 2 ]    let m = vec[1, 2]
m[1] = [ 3, 4 ] m[1] = vec[3, 4]
```

<https://n-jnvgavl4buxc3xao7d4lq2zxreghpugshyqyu5y-0lu-script.googleusercontent.com/userCodeAppPanel>

Page 1 of 5

6

- A. @100 = #(@200 1); @200 = #(3 4)
- B. @100 = #(1 @200); @200 = #(3 4)
- C. @100 = #(1 #(3 4))
- D. @100 = #(1 #(3 4)); @200 = #(3 4)
- E. @100 = #(1 2)
- F. @100 = #(1 2); @200 = #(3 4)

@100 = #(1 @200); @200 = #(3 4)

7

You gave the correct answer 🎉🎉🎉

8

Two arrays are created. So, there must be at least two things on the heap. This rules out **C** and **E**.

D is wrong because arrays refer values (e.g., 1, 2, and @200). [3, 4] is not itself a value; it's the *printing* of the value that resides at @200. **F** can be the heap after the second array is created. However, the subsequent mutation changes @100. So, **F** is wrong.

The mutation replaces the 1-th (i.e., second) element rather than the 0-th element, so **B** is correct, while **A** is wrong.

Python | 9

(Task 4 of 7) Which choice best describes the heap at the end of the following program?

Python [Run ▶]

Scala 3

```

v = [ 1, 2, 3 ]    val v = Buffer(1, 2, 3)
vv = [ v, v ]      val vv = Buffer(v, v)
vv[1][0] = 6       vv(1)(0) = 6
print(vv[0])       println(vv(0))

```

10

- A. @100 = #(1 2 3); @200 = #(@100 @100)
- B. @100 = #(1 2 3); @200 = #(@100 @300); @300 = #(6 2 3)
- C. @100 = #(1 2 3); @200 = #(#(1 2 3) #(6 2 3))
- D. @100 = #(1 2 3); @200 = #(1 2 3); @300 = #(6 2 3); @400 = #(@200 @300)
- E. @100 = #(1 2 3); @200 = #(6 @100)
- F. @100 = #(6 2 3); @200 = #(@100 @100)
- G. @100 = #(6 2 3); @200 = #(#(1 2 3) #(6 2 3))

```
@100 = #(1 2 3); @200 = #(6 @100)
```

11

The answer is `@100 = #(6 2 3); @200 = #(@100 @100)`.

12

Arrays refer values (e.g., 1 and `@200`). This rules out **C** and **G**.

Two arrays are created. So, there must be two arrays on the heap. This rules out **B** and **D**.

`vv` is bound to a 2-element array, and the 1-th element of the 2-element array must be the 3-element array. The mutation replaces the 0-th element in the 3-element array with 6. So, **F** is the correct answer, while **A** does not reflect the effect of the mutation, and **E** mutates the wrong array.

Python | 13

(Task 5 of 7) Which choice best describes the heap at the end of the following program?

Python [Run]

Scala 3

```
x = [ 3 ]          val x = Buffer(3)
v = [ 1, 2, x ]    val v = Buffer(1, 2, x)
x[0] = 4           x(0) = 4
print(v)           println(v)
```

14

- A. `@100 = #(3); @200 = #(1 2 @100)`
- B. `@100 = #(3); @200 = #(1 2 @300); @300 = #(4)`
- C. `@100 = #(4); @200 = #(1 2 @100)`
- D. `@100 = #(4); @200 = #(1 2 #(3))`
- E. `@100 = #(4); @200 = #(1 2 #(4))`
- F. `@100 = #(4); @200 = #(1 2 3)`
- G. `@100 = #(4); @200 = #(1 2 4)`

```
@100 = #(3); @200 = #(1 2 @100)
```

15

The answer is `@100 = #(4); @200 = #(1 2 @100)`.

16

D and **E** are wrong because arrays refer values. `[3]` and `[4]` are not values, although they can be the printed representation of a array.

F and **G** are wrong because the 2-th element of the 3-element array must be a array. The 3-element array is created by `[ 1, 2, x ]`. The value of `x` is a array at that

moment. This 3-element array is never mutated.

B is wrong because only two arrays are created. There must be at most two arrays on the heap.

A can be the heap after the two arrays are created. However, the subsequent mutation changes the shorter array. So, **C** is the correct answer.

Python | 

(Task 6 of 7) Which choice best describes the heap at the end of the following program?

Python 

Scala 3

```
x = [ 1, 0, 2 ]    val x = Buffer(1, 0, 2)
x[1] = x           x(1) = x
print(len(x))      println(x.length)
```

18

- A. @100 = #(1 @100 2)
- B. @100 = #(1 @200 2); @200 = #(1 0 2)
- C. @100 = #(1 #(1 0 2) 2)
- D. @100 = #(1 0 2)

@100 = #(1 @200 2); @200 = #(1 0 2)

19

The answer is @100 = #(1 @100 2).

20

C is wrong because arrays refer values. [1, 0, 2] is not a value, although it can be some array values printed.

B is wrong because only one array is created. There must not be two arrays on the heap.

D can be the heap after the array is created. But the subsequent mutation replaces the 1-th element of @100 with @100. So, **A** is the correct answer.

Python | 

(Task 7 of 7) Which choice best describes the heap at the end of the following program?

Python 

Pseudo

```
mv = [ 4, 5 ]    let mv = vec[4, 5]
print(mv[0])     print(mv[0])
```

22

- A. @100 = #(@200 @300); @200 = 4; @300 = 5
- B. @100 = #(4 5)
- C. @100 = #(4 5); @200 = 4

23

@100 = #(4 5)

24

You gave the correct answer 🎉🎉🎉

For now, the heap maps addresses only to arrays. This rules out **A** and **C**.

You have finished this tutorial 🎉🎉🎉

Please print the finished tutorial to a PDF file so you can review the content in the future. **Your instructor (if any) might require you to submit the PDF.**

Start time: 1761332798877